MODERN MONKLABS

Intern Assignment: PID Control

Physical AI | Concept understanding & technical documentation

What you need to do: Write a clear, beginner-friendly document that teaches **PID control** (the most common way robots and machines hold a target such as a speed, position or angle). Explain the idea in simple words, back it up with formulas, and prove it with graphs from **Python code you write yourself**. Write it so that someone new to the topic could learn PID from your document alone.
Time: 7 days. **Submit:** (1) one PDF document, max 15 pages; (2) a GitHub link with your Python code (use NumPy and Matplotlib; write the PID logic yourself, no ready-made control library) and a README explaining how to run it to recreate every graph.

Key words used below

Setpoint: the target value. **Error**: target minus actual value. **Plant**: the machine being controlled. **Gain** (Kp, Ki, Kd): how strongly each part of PID reacts. **Overshoot**: going past the target before coming back. **Step response**: how the system moves when the target suddenly changes, e.g. from 0 to 1 m.

1. Your document must have these 10 sections

#	Section	Must include
1	**The basic idea**	In simple words, explain how a controller keeps reducing the gap between the target and the actual value. Use a daily-life example (for example, adjusting shower water temperature). Draw a simple loop diagram: target → controller → machine → sensor → back to the start.
2	**The PID formula**	Write the formula, explain every letter and its unit, and explain what P (present error), I (past error) and D (how fast the error is changing) each do. Also show the step-by-step version a computer runs in small time steps.
3	**The test system (given)**	Simulate a **1 kg block on a spring with a damper** (like a car suspension): mass = 1 kg, damping = 10 N·s/m, spring = 20 N/m. The goal is to move it to **1 m**. Write its equation and explain why the spring needs a constant push to hold it there.
4	**How to judge a result**	Explain four measures: how fast it rises, how much it overshoots, how long until it settles down, and how far from the target it finally stops. Mark all four on one graph.
5	**Each part alone**	Try P only with at least 4 strengths, then add I, then add D, with a graph for each. Show **by calculation** why P alone always stops short of the target and by how much, and how D reduces bouncing. Check that your calculation matches your graphs.
6	**All together**	Compare P, PI, PD and full PID on one graph and in a table using the four measures. Choose your final gain values and explain why.
7	**Real-world problems**	Show each problem with a graph, then fix it: (a) the motor can push at most **30 N**, so the I part builds up and overshoots ("windup"); (b) a noisy sensor (±1 mm) makes the D part jumpy; (c) a sudden target change gives a huge D spike ("kick"); (d) what happens if the computer updates too slowly.
8	**Choosing the gains**	Describe how you would tune by hand, step by step. Then use the **Ziegler–Nichols** recipe on a second system, $1/(s+1)^3$: raise P until it swings steadily, note that gain and the swing period, apply the recipe table, plot the result and improve it.
9	**The code**	Write a reusable PID in Python that includes the fixes from section 7, run it on the test system, and show the output.
10	**Real robots & myths**	Give at least 4 real examples (motor speed, drone, robot arm, line follower, car…) and how PID is used in each. List at least 4 common wrong beliefs about PID and correct them.

2. How to write it

• Simple explanation first, then the formula, then the graph. • Every graph needs a title, labelled axes with units, a legend and one line saying what it shows. • Use numbers, not vague words: write "overshoot dropped from 40% to 4%", not "much better". • Anyone should get the same graphs by running your code.

3. Rules

You may use AI tools; mention briefly how you used them. In a **30-minute interview** we will change something (a gain, the weight, the noise or the force limit) and ask you to **predict what will happen before** running your code, then explain the result. If you cannot finish everything, submit what you have with a note on what is missing.

4. How you will be evaluated

Correct concepts	Calculations & code	Easy to understand	Graphs & writing quality	Interview explanation
25%	20%	20%	15%	20%